

Arduino platform simulator

Adam Balko

Supervisor: RNDr. David Bednárek, Ph.D.

Arduino simulation

Testing a code for an **Arduino** machine can be fairly difficult. While debugging options exist, they usually rely on connecting to the physical hardware, sometimes requiring additional Arduino modules and libraries. For virtual environments, the usual choice is to test the code with an emulator, that processes instructions for the AVR or ARM chip on the board.

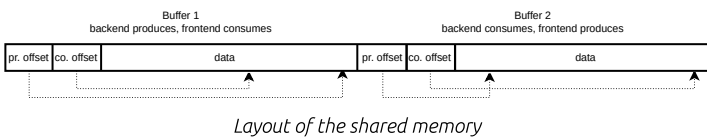
Goal of this work is to skip the **emulation** process and create a working Arduino simulator by mimicking the changes of digital values on the board's pinout. The user code will be compiled directly to the *x86-64 architecture* instruction set, and thus able to run at the native speed of the host machine.

We want to show the effects of the program on a user-defined digital circuit in a **GUI**. The user can interact with the circuit via buttons or other custom components. At the same time the user program should be fully *debuggable*, while the GUI stays responsive.

Inter-process communication

In order for the two processes to exchange data and stay synchronised, they both produce **events**. These consist of a simple identifier (such as *write*, *pinmode* etc.) and up to 3 parameters (e.g. *pin* and a *value*). These are encoded into a text **message** and sent to the other process via *Inter-process communication (IPC)*. We implemented 2 most common methods of IPC:

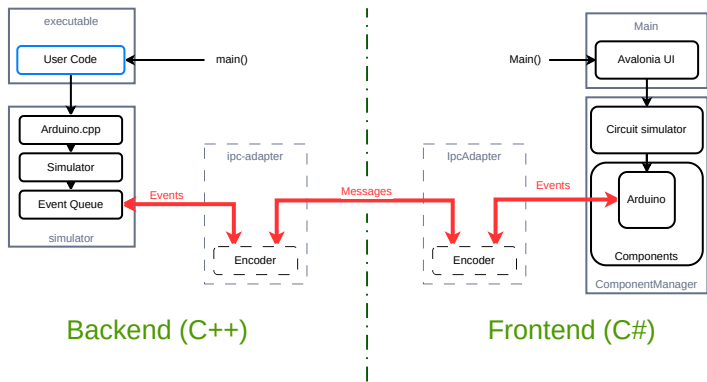
- Through **TCP connection** on the localhost, where backend is the server and frontend is the client.
- Through **shared memory**, where we create two *circular buffers*. The backend writes messages into one buffer and reads from the other. Frontend does the same but in reverse.



2 process architecture

By default, common debuggers operate in an *"all-stop"* mode where the target process must be completely paused to perform any meaningful interaction. If the GUI has been part of the same process as the systems running the simulation, it would *freeze* and become unresponsive every time a breakpoint is hit. The only way to avoid this behavior is to separate the GUI to a **different process**.

The **C++ backend** provides the Arduino library functions, compiles together with the input program and executes it. The **C# frontend** receives events produced by the backend, displays them and updates the circuit the Arduino is connected to. It also sends events back on human interaction with the circuit.



Top level architecture of the 2 processes

Components and Scenes

In frontend, the circuit itself is a bipartite graph where the partitions are **components** and **electrical nodes**, and the edges are connected **pins**. A pin can *drive* a digital value to the node, notifying all connected pins about the change, making a cascading effect.

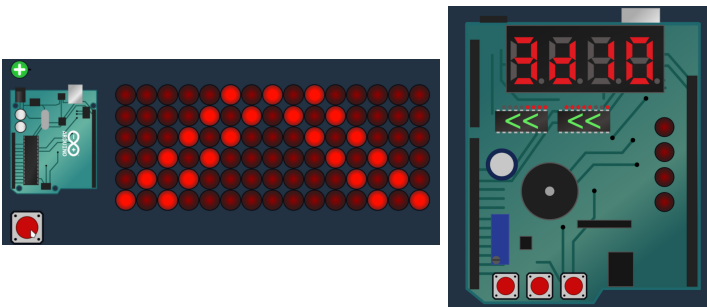
The user should be able to define their own components, like buttons, LEDs, registers, etc. We require 3 types of files for 1 component *type*:

- A **YAML configuration file** contains pins and metadata information
- SVG sprites** are rendered by the *Avalonia UI* framework
- C# classes** define the logic, by implementing the abstract *Component* class. It provides methods for changing the sprites and values on the pins. The abstract methods get called on user interactions, changed values on the nodes etc.

A **scene** is another YAML file that defines the component *instances*, where and how they are rendered, and which of their pins are connected together into electrical **nodes**.

Conclusion

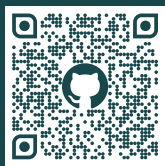
In the end we managed to develop two nontrivial applications. One for running and simulating the user's Arduino code. The other one for creating an interactive virtual environment. With the implemented events and scene creation tools, we could model the **Funshield** board used during MFF courses. Proving it is built on a solid foundation, ready for new quality of life features and optimizations.



Scene with a chevron drawing program (left) and Funshield scene (right)



Faculty of Mathematics
and Physics
Charles University



9.9.2026
balko.nitra@gmail.com
<https://github.com/TenIdiotZInternetu/Arguino/tree/master>